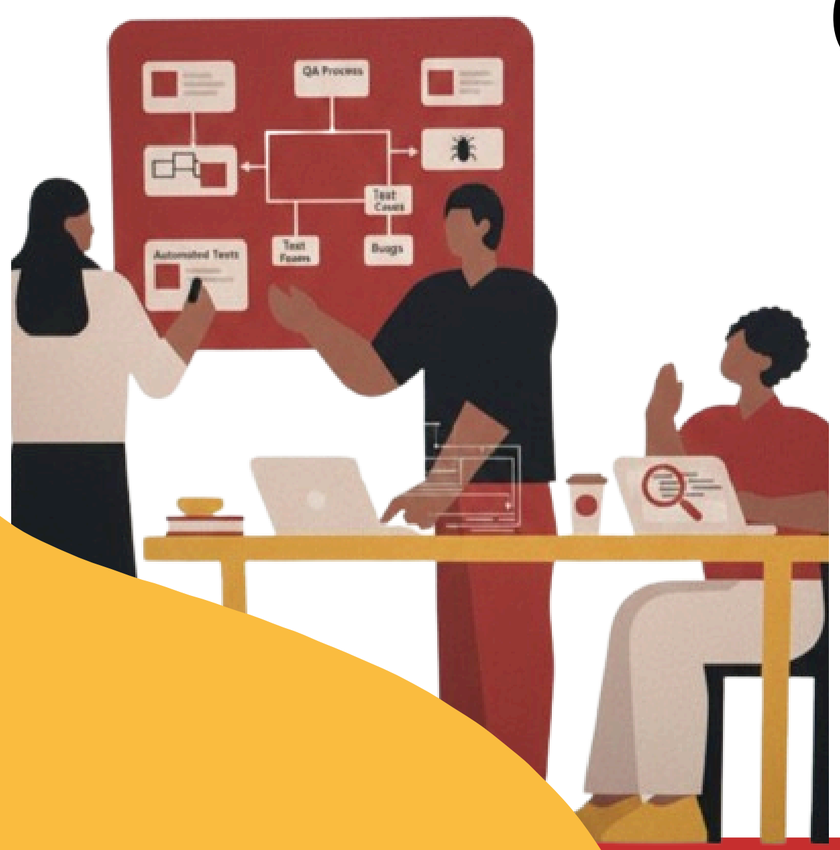


QUALITY ASSURANCE (QA)

&

SOFTWARE TESTING

ICT Upskilling Program-HTU
Prepared by: Rania Hasan
Feb 2026

ABOUT ME

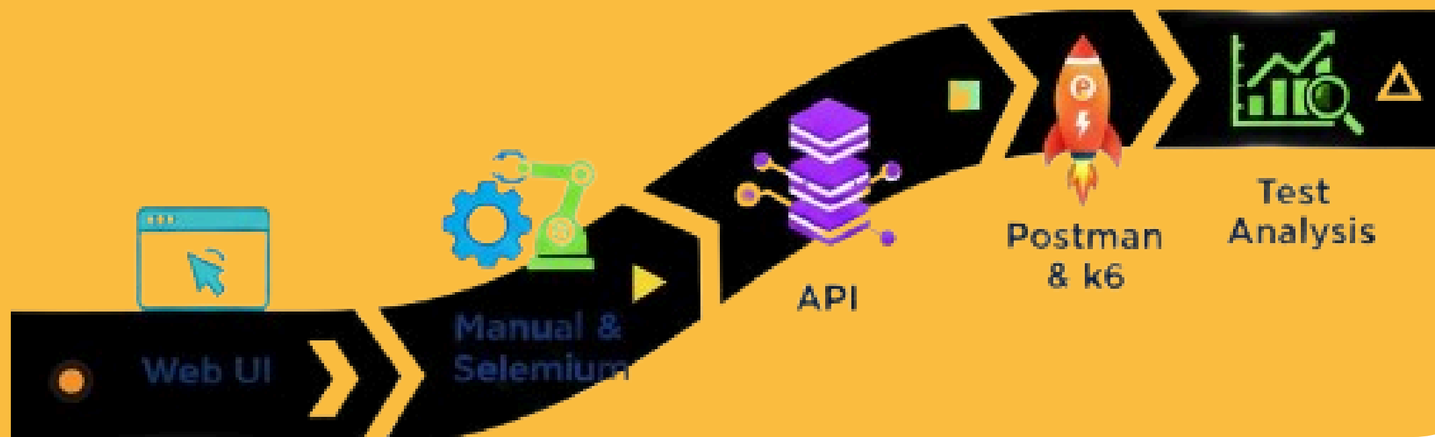
I’m Rania, a Management Information Systems graduate from the University of Jordan with a very good GPA. I have a passion for software testing and enjoy ensuring that applications work smoothly, identifying bugs, and improving the user experience. I love manual testing, exploring APIs, and automating UI tests, and I’m always eager to learn new tools and apply them in real projects.

PROJECT OVERVIEW

The project focuses on testing web and backend applications using manual testing,
UI automation, API testing, and performance testing.

We used SauceDemo for web testing to validate user flows like login, browsing, cart, and checkout.
DummyJSON was used for API testing to check endpoints, responses, and performance.

The goal was to practice a full QA cycle across different systems,
applying testing methods and tools to improve software quality.



TEST PLAN

How I Created the Test Plan:

1. Understand the System

- Explored the website and identified key features: login, browse products, add to cart, checkout.

2. Define Test Scenarios

- Created scenarios for Happy Paths (expected behavior) and Negative Paths (invalid inputs or edge cases).

3. Write Test Cases

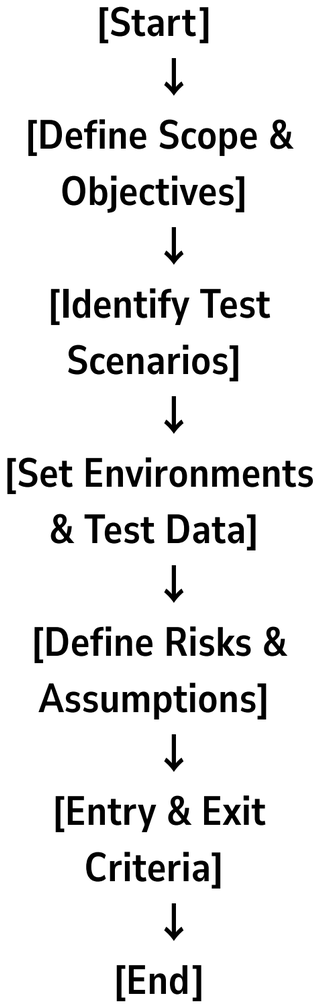
- Documented each scenario with steps, expected results, and evidence placeholders.

4. Follow QA Criteria

- Ensured coverage of functional flows, edge cases, and clear reproducible steps.

5. Example (Test Scenario):

- Happy Path: User logs in with correct credentials → redirected to home page.
- Negative Path: User enters invalid password → error message displayed, login blocked.



Title: Prevent Checkout with Empty Fields

EX.(TEST CASE)

Steps:

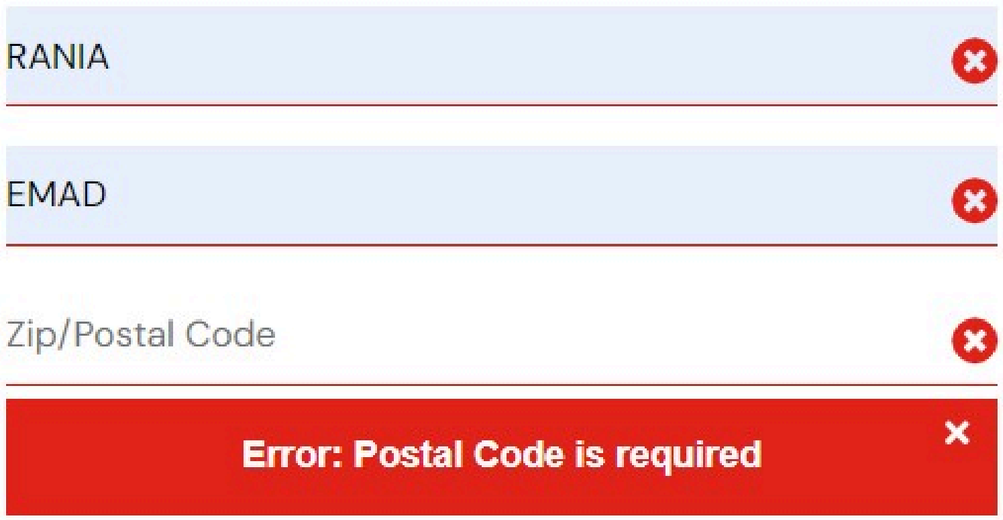
1. Go to the checkout page
2. Leave all fields empty
3. Click Continue.

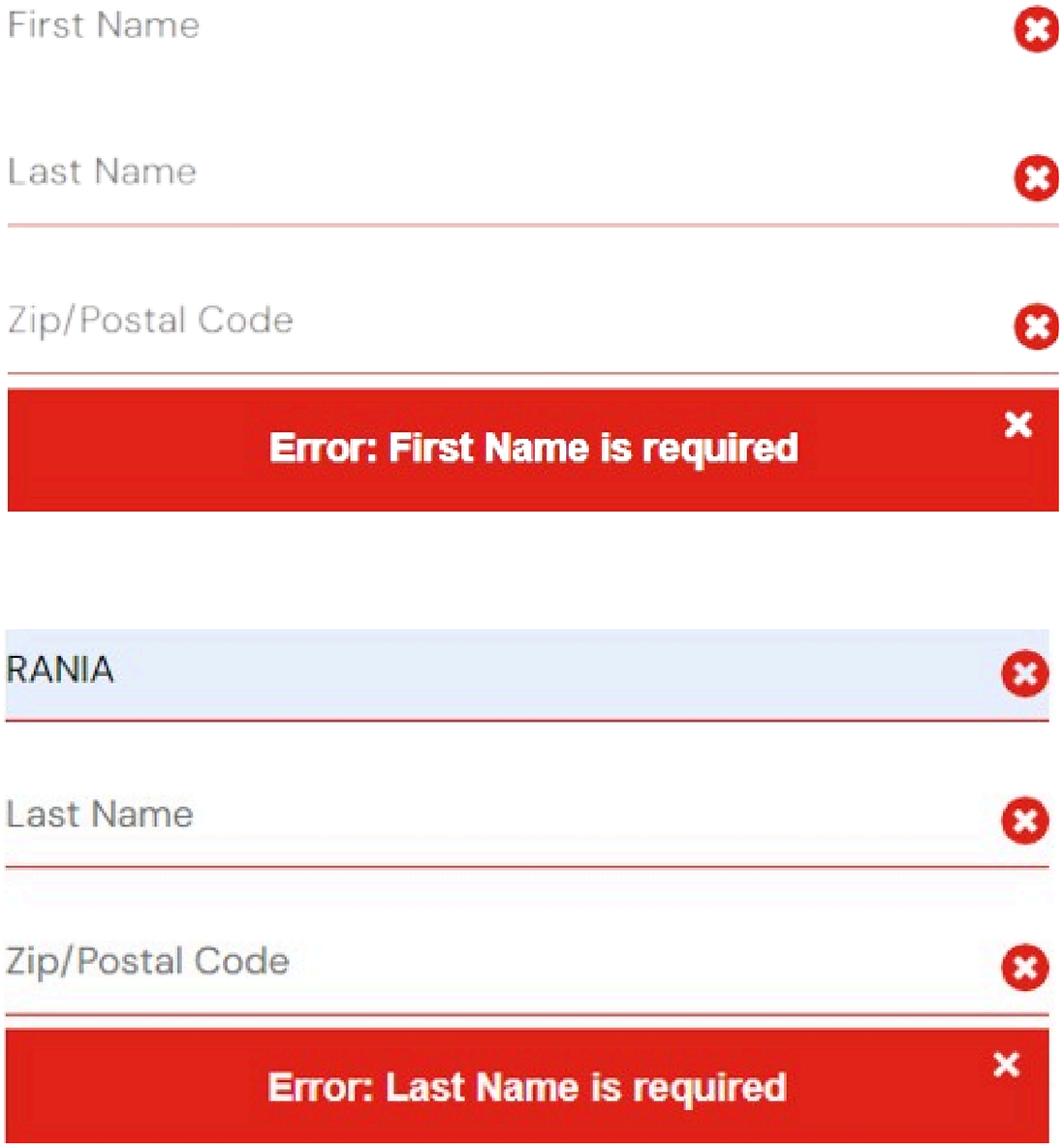
Expected:

Show errors, block submission

Actual: Errors displayed, user stays on page

Status: ✅ Pass



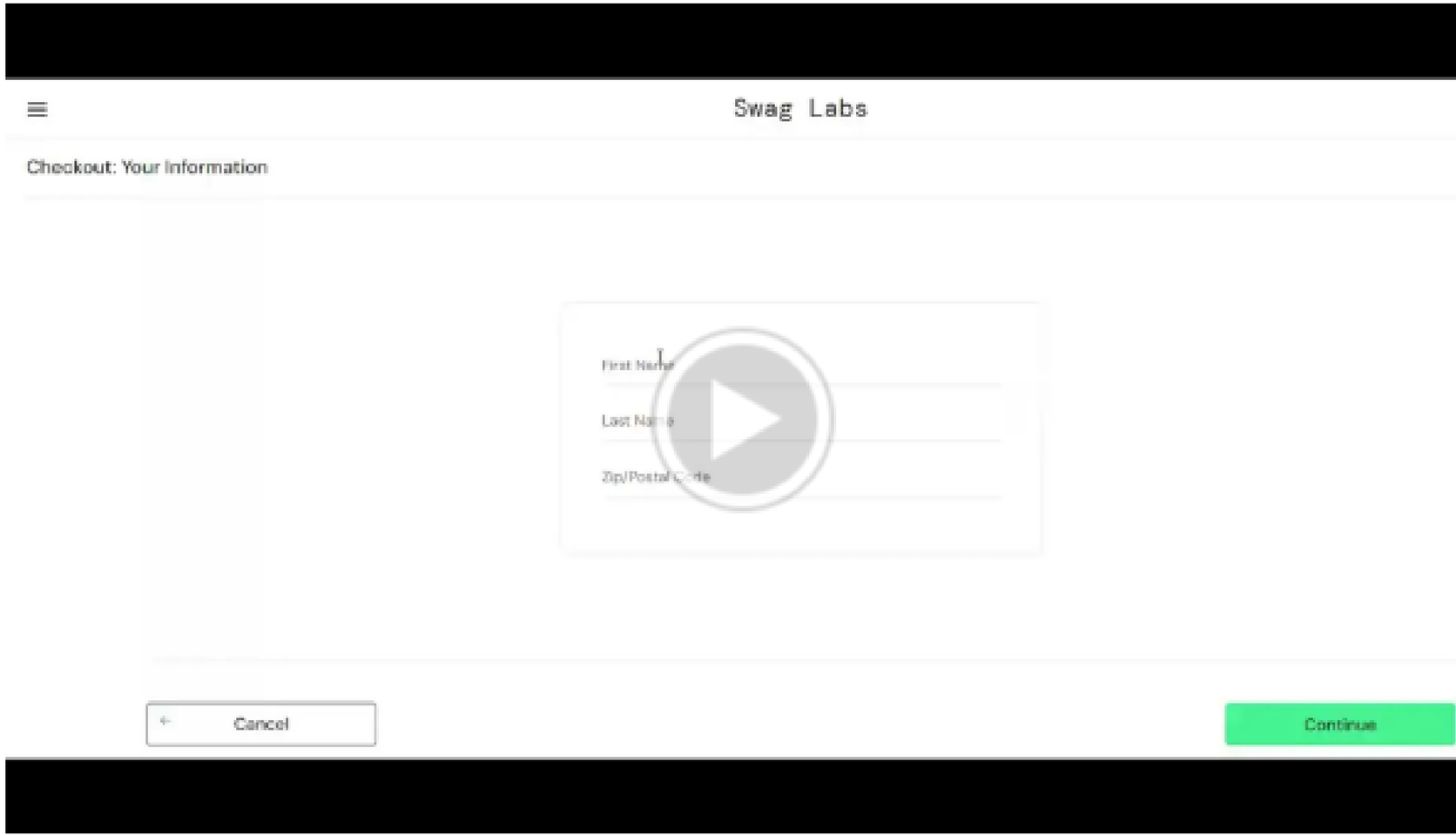


Title: Reject Whitespace-Only Input on Checkout

EX.(TEST CASE)

Steps:

1. Go to the checkout page
2. Enter only whitespace in all fields
3. Click Continue



Expected:

Show error, block submission

Actual: Accepted input, moved forward

Status: ❌ Failed

BUG REPORT

Field	Details
ID	Bug-009
Title	Checkout form accepts whitespace-only input in required fields
Severity	Critical
Priority	Immediate
Steps	1. Navigate to the Checkout page. 2. Enter only whitespace (e.g., " ") in First Name, Last Name, and Zip/Postal Code fields. 3. Click the Continue button.
Expected	1. System should reject whitespace-only input. 2. Error messages should be displayed for each required field. 3. Form submission should be blocked until valid data is entered.
Actual	1. System accepts whitespace-only input. 2. Checkout continues without validation
Evidence	https://drive.google.com/file/d/1t11x27HjlhVpLfM9h48x8PXRx2tWvMlw/view?usp=sharing

UI AUTOMATION TESTING (SELENIUM + TESTNG)

```java
import java.io.File;

public class MyTestCases {

    String myWebsite = "https://www.saucedemo.com/";
    WebDriver driver = new EdgeDriver();

    String userName = "standard_user";
    String password = "secret_sauce";
    Random rand = new Random();

    @BeforeTest
    public void mySetUp()  {
        driver.get(myWebsite);
        driver.manage().window().maximize();

        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

    }

    @Test(priority = 1)
    public void login() throws InterruptedException {
        driver.findElement(By.id("user-name")).sendKeys(userName);
        driver.findElement(By.id("password")).sendKeys(password);
        driver.findElement(By.id("login-button")).click();
        WebElement productsTitle = driver.findElement(By.className("title"));
        Assert.assertEquals(productsTitle.getText(), "Products",
                "Login failed: Products page not displayed");

    }
```

Problems | Javadoc | Declaration | Console | Results of running class MyTestCases | Coverage

Purpose:
- Validate critical user workflows
- Reduce regression risks

Tool & Framework:
- Selenium WebDriver + TestNG (Java)
- Stable, flexible, and maintainable

Automated Scenarios:
- Login / Logout / Invalid Login
- Product sorting & cart operations
- Checkout process

Approach & Structure:
- Sequential, priority-based test execution
- Centralized setup & teardown
- Reusable locators and assertions for validation

Results:
- 8 test cases executed end-to-end
- Navigation, messages, and expected behavior verified
- Screenshots captured for failures

API TESTING WITH POSTMAN

Where the world builds APIs

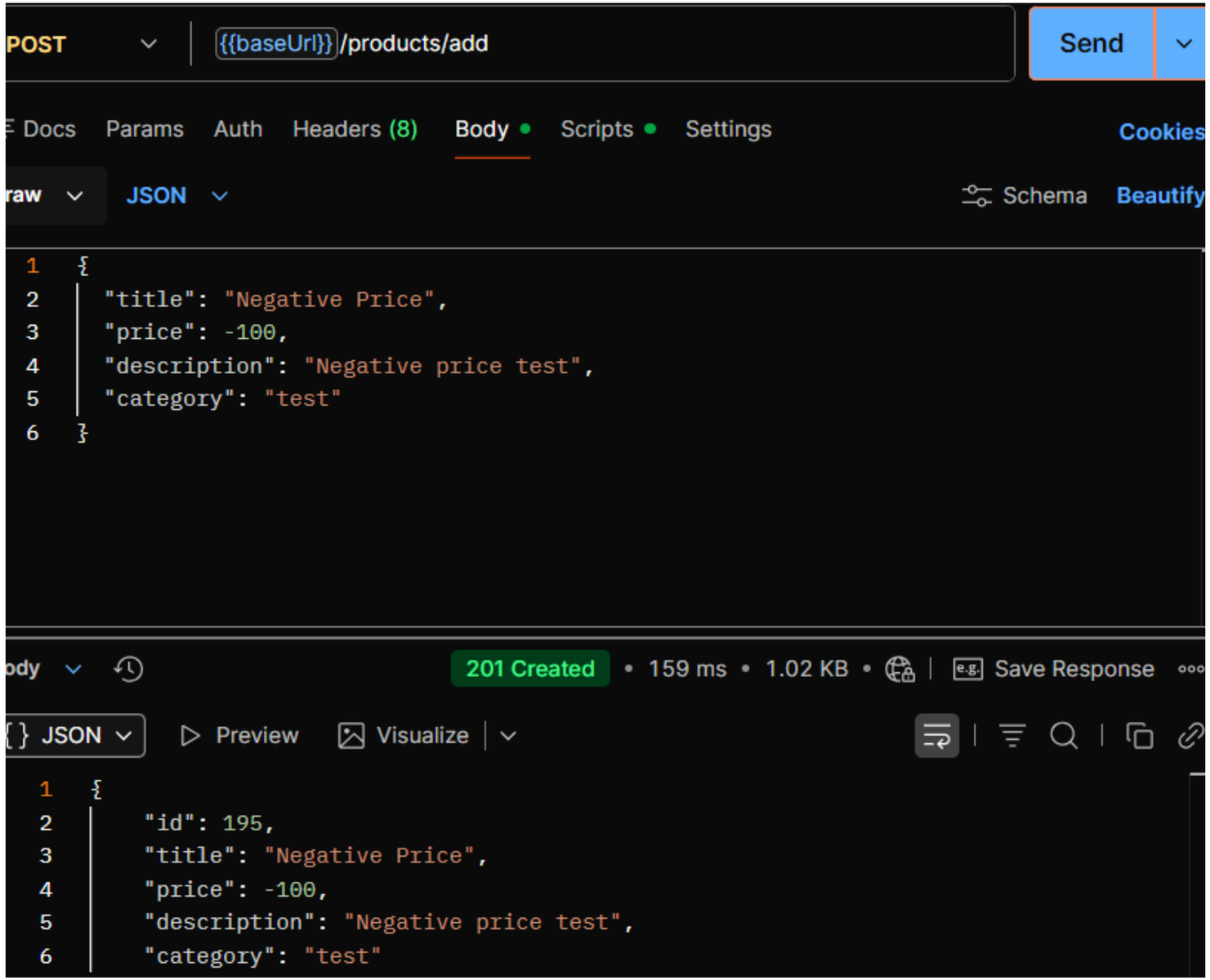
Unify API design, testing, documentation, monitoring, and discovery on one platform that integrates with the rest of your stack, including every major gateway and Git solution

- Role of Postman: Used to test REST APIs manually and automatically, ensuring requests return expected results.

- Requirements: Cover at least 20 requests including GET, POST, authentication, and negative scenarios.

- How it helped: Allowed me to validate API functionality, catch potential errors early, and generate reusable, maintainable test collections and reports.

EX. POSTMAN BUG FOUND IN DUMMYJSON



While testing with Postman, I applied data-driven negative scenarios and discovered that the API accepts invalid prices like (0,-100,abc,””,)), which is a critical bug affecting data integrity.

Context:

- Tested POST /products/add with invalid product prices like -100.

K6 PERFORMANCE TEST ANALYSIS

Overview:

• Performance test on API with up to 40 virtual users.

• Duration: 2 minutes + smoke test for stability.

Observations:

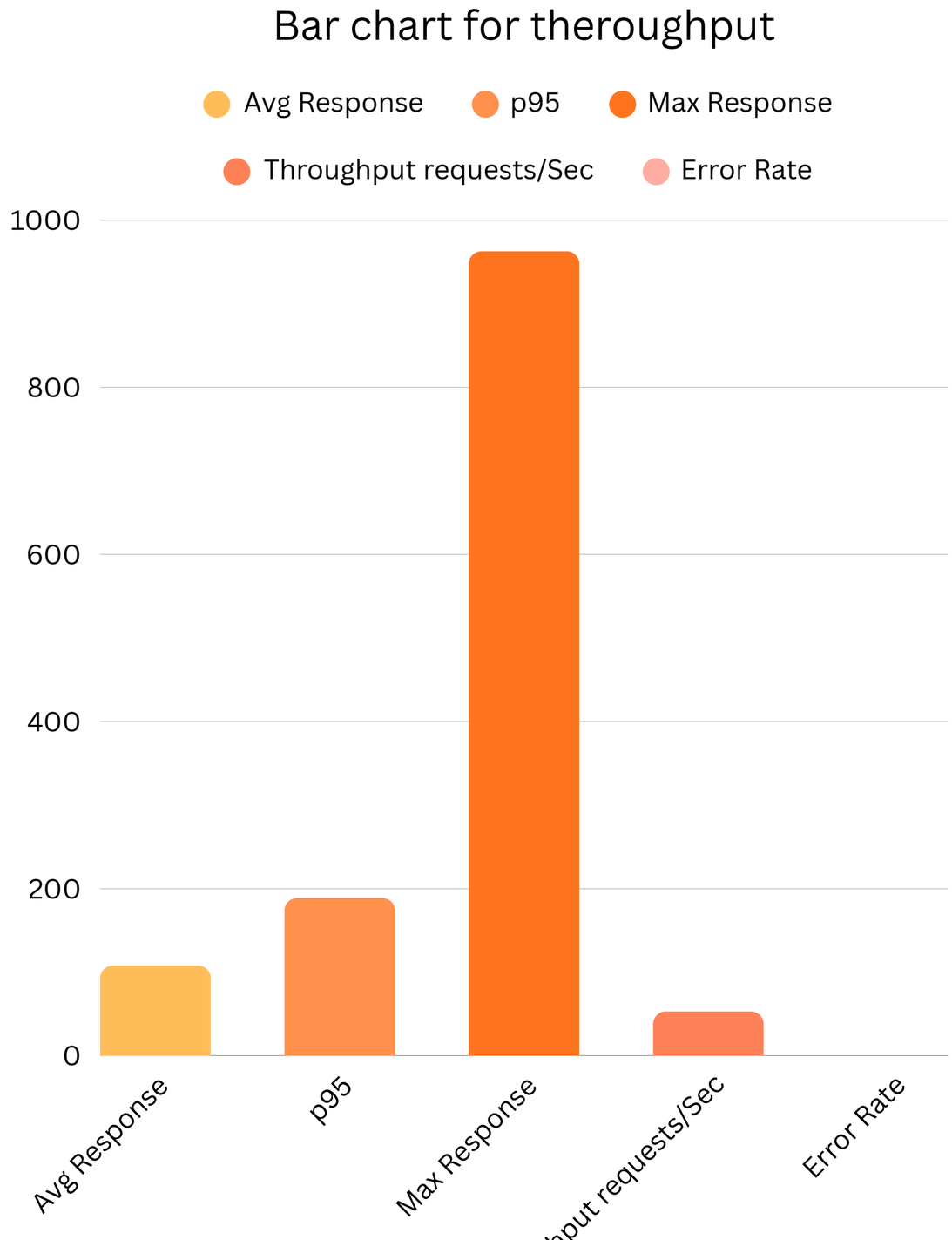
• System remained responsive under moderate load.

• Throughput stable, no errors.

• Occasional slower requests → backend or network.

Recommendations:

• Add caching, optimize backend, consider load balancing.



THANK YOU...

ANY QUESTIONS?